ERKLÄRVIDEO 3

GROUP BY und HAVING – so wie im Abitur



Auswertungsreihenfolge, GROUP-BY-Regel, WHERE gegen HAVING und COUNT gegen SUM

Akte 3 · DS 5–6 · 10–12 min

Die Reihenfolge, in der die Datenbank denkt

1. **FROM** – Tabellen (bei mehreren: Kreuzprodukt)
2. **WHERE** – einzelne Zeilen filtern
3. **GROUP BY** – Zeilen in Gruppen (Fächer) sortieren
4. **HAVING** – ganze Gruppen filtern
5. **SELECT** – je Gruppe eine Ergebniszeile berechnen
6. **ORDER BY**, **LIMIT** – sortieren, abschneiden

EISDIELE (GASTMODUS)

Sorte(SorteNr, Name, Preis, Vegan)

Verkauf(VerkaufNr, ↑SorteNr, Datum, Kugeln)

Gruppieren: eine Zeile je Tag

```
SELECT Datum, COUNT(*) AS Verkaeufe, SUM(Kugeln) AS Kugeln
FROM Verkauf
GROUP BY Datum
```

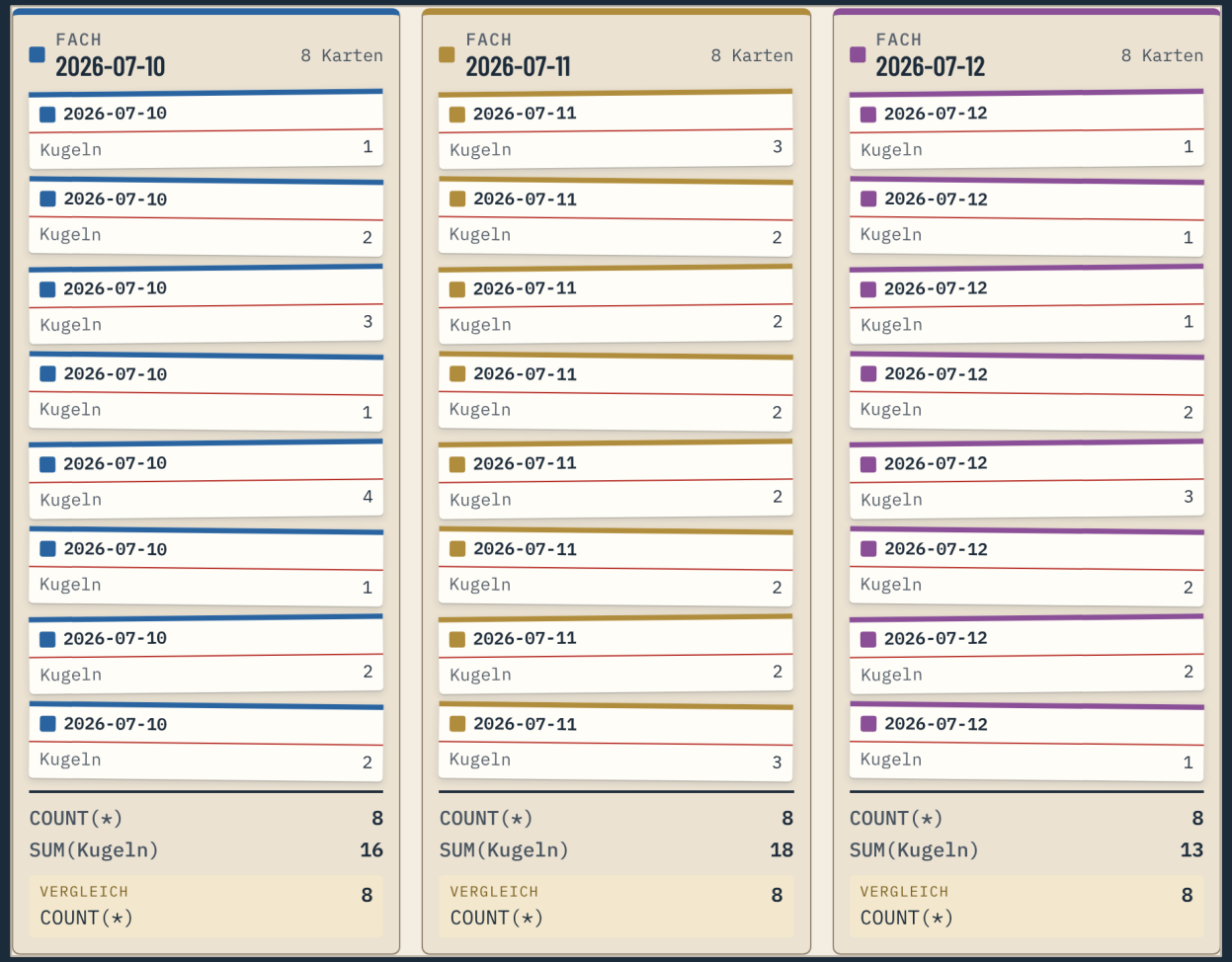
ERGEBNIS · 7 ZEILEN

Datum	Verkaeufe	Kugeln
2026-07-06	9	21
2026-07-07	8	14
2026-07-08	8	16
2026-07-09	8	20
2026-07-10	8	16
2026-07-11	8	18
2026-07-12	8	13

AGGREGATFUNKTIONEN

- `COUNT(*)` zählt Zeilen der Gruppe
- `SUM`, `AVG`, `MIN`, `MAX` rechnen mit einer Spalte
- eine Gruppe → **eine** Ergebniszeile

In den Sortierfächern



EIN FACH JE DATUM

Hier Freitag bis Sonntag (`WHERE Datum >= '2026-07-10'`): jeder Verkauf landet im Fach seines Tages.

COUNT zählt die Karten im Fach, SUM addiert die Kugeln darauf.

Die GROUP-BY-Regel

✘ FALLE P3

```
SELECT Datum, Kugeln, COUNT(*)
FROM Verkauf
GROUP BY Datum
```

✔ RICHTIG

```
SELECT Datum, SUM(Kugeln) AS Kugeln, COUNT(*) AS Verkaeufe
FROM Verkauf
GROUP BY Datum
```

REGEL (EH §8)

Jede Spalte im SELECT ist **aggregiert** oder steht im **GROUP BY**.

Welche „Kugeln" soll die Zeile vom 6. Juli zeigen – es gibt neun Werte!

Gruppieren nach dem Schlüssel

```
SELECT Sorte.SorteNr, Sorte.Name, COUNT(*) AS Verkaeufe
FROM Sorte, Verkauf
WHERE Sorte.SorteNr = Verkauf.SorteNr
  AND Verkauf.Kugeln >= 2
  AND Sorte.Preis >= 1.7
GROUP BY Sorte.SorteNr, Sorte.Name
```

ERGEBNIS · 5 ZEILEN

SorteNr	Name	Verkaeufe
5	Haselnuss	3
6	Sanddorn-Mango	2
7	Stracciatella	6
8	Karamell	4
10	Schoko-Minze	3

WARUM SORTENR?

Nach dem **Primärschlüssel** gruppieren – Namen können doppelt sein (P3).

Name steht zusätzlich im GROUP BY, damit er im SELECT stehen darf.

WHERE filtert Zeilen, HAVING filtert Gruppen

```
SELECT Datum, COUNT(*) AS Grosse
FROM Verkauf
WHERE Kugeln >= 3
GROUP BY Datum
HAVING COUNT(*) >= 2
```

ERGEBNIS · 6 ZEILEN

Datum	Grosse
2026-07-06	2
2026-07-07	2
2026-07-08	2
2026-07-09	3
2026-07-10	2
2026-07-11	2

ZWEI FILTER

- `WHERE Kugeln >= 3`: jeder **Verkauf** einzeln, vor dem Gruppieren
- `HAVING COUNT(*) >= 2`: jeder **Tag** als Ganzes, danach
- der 12. Juli hat nur einen großen Verkauf → fällt raus

In der Zeitlupe: der Schritt HAVING

6
GRUPPEN
vorher 7

HAVING COUNT(*) >= 2

HAVING prüft ganze Gruppen: 6 von 7 Gruppen erfüllen die Bedingung, 1 fällt komplett weg.

2026-07-06 2 Zeilen · 2026-07-07 2 Zeilen · 2026-07-08 2 Zeilen · 2026-07-09 3 Zeilen · 2026-07-10 2 Zeilen · 2026-07-11 2 Zeilen · ~~2026-07-12~~ 1 Zeile **fällt weg**

GRUPPE	VERKAUFNR	SORTENR	DATUM	KUGELN
2026-07-06	2	1	2026-07-06	5
2026-07-06	8	1	2026-07-06	4
2026-07-07	12	2	2026-07-07	3
2026-07-07	14	1	2026-07-07	3
2026-07-08	19	8	2026-07-08	3
2026-07-08	20	1	2026-07-08	4
2026-07-09	26	1	2026-07-09	5
2026-07-09	28	3	2026-07-09	3
2026-07-09	32	1	2026-07-09	3
2026-07-10	36	2	2026-07-10	3
2026-07-10	38	1	2026-07-10	4
2026-07-11	42	5	2026-07-11	3
2026-07-11	49	1	2026-07-11	3
~~2026-07-12~~	~~54~~	~~1~~	~~2026-07-12~~	~~3~~

ZEITLUPE

Klausel für Klausel in Auswertungsreihenfolge.

Bei HAVING wird die ganze Gruppe grau – nicht einzelne Zeilen.

Kurz anhalten: Jetzt du!

⏸ **VIDEO ANHALTEN**

Welche Sorten wurden am **Wochenende** (ab **2026-07-11**) **mindestens zweimal** verkauft? Gib SorteNr, Name und die Anzahl der Verkäufe aus – die häufigste Sorte zuerst.

```
Sorte(SorteNr, Name, Preis, Vegan)
Verkauf(VerkaufNr, ↑SorteNr, Datum, Kugeln)
```

TIPP

Was ist eine Zeilenbedingung, was eine Gruppenbedingung?

Wonach wird gruppiert?

Auflösung

```sql
SELECT Sorte.SorteNr, Sorte.Name, COUNT(*) AS Verkaeufe
FROM Sorte, Verkauf
WHERE Sorte.SorteNr = Verkauf.SorteNr
  AND Verkauf.Datum >= '2026-07-11'
GROUP BY Sorte.SorteNr, Sorte.Name
HAVING COUNT(*) >= 2
ORDER BY Verkaeufe DESC
```

ERGEBNIS · 5 ZEILEN

SorteNr	Name	Verkaeufe
7	Stracciatella	4
1	Vanille	3
3	Erdbeere	2
5	Haselnuss	2
8	Karamell	2

TYPISCHE FEHLER

- Datum im HAVING (P1)
- `HAVING Verkaeufe >= 2` – Alias im HAVING (P1)
- `SUM(Kugeln)` statt `COUNT(*)` (P4)

Die Fallen bei GROUP BY

P1 **WHERE ↔ HAVING**
Aggregat im WHERE (`WHERE COUNT(*) >= 2` geht nicht), Zeilenbedingung im HAVING, Alias im HAVING.

P3 **GROUP-BY-Regel**
Spalte im SELECT weder aggregiert noch gruppiert; nach Name statt nach Schlüssel gruppiert.

P4 **COUNT ↔ SUM**
„Anzahl Verkäufe" = `COUNT(*)`, „Anzahl Kugeln" = `SUM(Kugeln)`. `COUNT(Spalte)` zählt keine NULL-Werte.

✘ P1

```
SELECT Datum, COUNT(*)
FROM Verkauf
WHERE COUNT(*) >= 9
GROUP BY Datum
```

Das nimmst du mit

ERMITTLER-NOTIZ

- FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- Eine Gruppe ergibt genau eine Ergebniszeile.
- Jede SELECT-Spalte ist aggregiert oder steht im GROUP BY – gruppiere nach dem Schlüssel.
- WHERE filtert Zeilen vor dem Gruppieren, HAVING filtert Gruppen danach – ohne Alias.
- COUNT(*) zählt Zeilen, SUM(Spalte) addiert Werte.

Weiter im Lab

SPUR IM LAB

- **Ermittlung 3.1 „Kassensturz“** · A3-E1-01 … 08 · Werkzeug: Sortierfächer
- **Ermittlung 3.2 „Die Statistik lügt nicht“** · A3-E2-01 … 08 · Werkzeug: Zeitlupe

ARBEITSBLATT UND TRAINING

- AB 3.1 „Kassensturz“ und AB 3.2 „Die Statistik lügt nicht“
- Verhör 3 nach Freischaltung durch die Lehrkraft

NÄCHSTES VIDEO

Video 4 – Redundanz, Anomalien und gute Schlüssel